

We'll define and initialize the variables for the name, password, and balance, and build methods to implement each of the operations. We should then be able to instantiate any number of `Account` objects. Like the initial class from Chapter 1, this is a simplified `Account` class that only uses integers for the balance and keeps the password in cleartext. While you wouldn't use simplifications like these in a real banking application, they will allow us to concentrate on the OOP aspects involved.

The new code for the `Account` class is presented in [Listing 4-1](#).

## File: Account.py

---

```
# Account class

class Account():
    ❶ def __init__(self, name, balance, password):
        self.name = name
        self.balance = int(balance)
        self.password = password

    ❷ def deposit(self, amountToDeposit, password):
        if password != self.password:
            print('Sorry, incorrect password')
            return None

        if amountToDeposit < 0:
            print('You cannot deposit a negative amount')
            return None

        self.balance = self.balance + amountToDeposit
        return self.balance

    ❸ def withdraw(self, amountToWithdraw, password):
        if password != self.password:
            print('Incorrect password for this account')
            return None

        if amountToWithdraw < 0:
            print('You cannot withdraw a negative amount')
            return None

        if amountToWithdraw > self.balance:
            print('You cannot withdraw more than you have in your account')
```